Aarhus University

Digital Circuits

Laboratory Report 3

Author 1:
Henrik Hoangkhanh Huynh
202307381

Auther 2:
Jakob Lund
202007550

Department of Electrical and Computer Engineering

December 10 2023

Contents

1 Introduction **1**

2 Theory **2**

2.1 Counter 2

2.1.1 Asynchronous Counters 2

2.1.2 Synchronous Counters 2

2.2 Binary Counter 2

2.3 Flip-Flops 3

2.3.1 Latches 3

2.3.2 JK Flip-Flop 3

2.4 States 3

3 Experiment **4**

3.1 Truth table 4

3.2 Setup 4

3.2.1 JK flip-flop 4

3.2.2 Counter 5

3.3 Diagram 6

3.4 Simulation 7

3.5 Implementation 7

4 Conclusion **8**

1 Introduction

In this laboratory exercise, we will be exploring the design and implementation of a digital counter. We start with the theory behind different types of counters, such as asynchronous and synchronous counters, and how they can be implemented alongside flip-flops to make a circuit that counts from 0 to 9 and back down to 0 in a continuous loop. To see the output of the counter, we use a 7-segment display, which is also described in the theory section.

In the experiment section, we create a truth table that helps with understanding and designing the circuit for the counter. We will also run a simulation of said circuit to confirm that it works as intended.

2 Theory

2.1 Counter

A counter is a device that can increment or decrement a value based on a clock signal. Counters can be either asynchronous or synchronous.

2.1.1 Asynchronous Counters

Asynchronous counters are counters where the flip-flops are clocked at different times. The clock pulse can ripple through the circuit (hence the name of the ripple counter), and the output of an asynchronous counter changes at different times, depending on the propagation delay of each flip-flop. Examples of asynchronous counters include the ripple counter and the Johnson counter.

2.1.2 Synchronous Counters

Synchronous counters are counters where all the flip-flops are clocked at the same time. The output of a synchronous counter changes in parallel with the clock signal. They can operate on higher frequencies than asynchronous counters because they do not have cumulative delay due to the same clock being given to each flip-flop. Though they are more efficient, they are also more expensive to implement, since they have to withstand the higher frequencies. Examples of synchronous counters include the binary counter and the decade counter.

2.2 Binary Counter

A binary counter is a counter that counts from one integer to its following integer in binary, which can be in both the positive and negative direction based on an input signal. An example is that from 0001 (1), the following integer will be 0010 (2) in the positive direction and 0000 (0) in the negative direction.

2.3 Flip-Flops

Flip-flops are circuits that can store states. Different flip-flops can determine the next state based on current states and input signals. The states are updated or "loaded" when a clock signal is given. There are two ways flip-flops may react to clock signals, when the signal goes to high (rising edge) or when the signal goes to low (falling edge). Examples of different flip-flop types are: JK flip-flop, D flip-flop, and T flip-flop, which all work in different ways. The only flip-flop used in this experiment is the JK flip-flop, so this is where the focus will be.

2.3.1 Latches

Another circuit that is able to store states is the latch. Unlike the flip-flops, latches do not necessarily need to be connected to a clock, but are instead connected to an enable signal. This is because they are not edge sensitive but instead level sensitive. Like the flip-flops, there a many different latches such as the S-R latch and the D-latch. Since a latch can store memory and be connected to a clock through its enable input, you can make a flip flop out of a latch. An example of this is a master slave D flip-flop, which is made from two D-latches.

2.3.2 JK Flip-Flop

A JK flip-flop is a flip-flop that has a J and K input, a clock alongside a Q and complementary $\bar{Q}$ output. The JK flip-flop works like an SR flip-flop but with more functionality. Like the SR flip-flop, the JK flip-flop can set (state to 1) and reset (state to 0) the state, but the JK flip-flop also has a state that inverts/complements the current state. This behaviour is shown in table 2.1.

J	K	Q(t+1)
0	0	Q
0	1	0
1	0	1
1	1	$\bar{Q}$

Table 2.1: Truth table of a JK flip-flop

2.4 States

The algebraic notation where an output is written as a function of time (t) used in table 2.1 is part of what is called state equations. These state equations are used to describe what happens to an output from one state Q(t) to the next $Q(t+1)$, where each state represents a specific value. A one bit state machine for instance only has two possible states: 0 and 1.

3 Experiment

3.1 Truth table

The truth table for the counter is used to make a design where it counts from 0 to 9 and then back to 0. The truth table is shown in table 3.1. On the left side of the table is the current state of the counter, in the middle is the JK flip flop, and on the right side is the next state. The reason why the truth table only goes to 9 and not 15 (1111 in 2-bit) is because all states from 10 to 15 are "do not care".

Q_4	Q_3	Q_2	Q_1		J	K		X
0	0	0	0		1	x		$\bar{Q}$ or 1
0	0	0	1		0	0		Q
0	0	1	0		0	0		Q
0	0	1	1		0	0		Q
0	1	0	0		0	0		Q
0	1	0	1		0	0		Q
0	1	1	0		0	0		Q
0	1	1	1		0	0		Q
1	0	0	0		0	0		Q
1	0	0	1		x	1		$\bar{Q}$ or 0

Table 3.1: Truth table for the counter.

3.2 Setup

3.2.1 JK flip-flop

The JK flip-flop is used as the memory element in the counter and the pin-out is shown in figure 3.1. This JK flip-flop chip contains two JK flip-flops, but only one is used in this experiment. Pin 1 is the clock, pin 2 is the reset input, pin 3 is the K input, pin 14 is the J input, and pin 12 is the output, Q. Power is supplied to pin 4 and ground is connected to pin 11. Pins 5 to 10 are not used as they belong to the second flip-flop. Here, the reset input is connected to power, so the flip-flop does not reset (because it is active low).

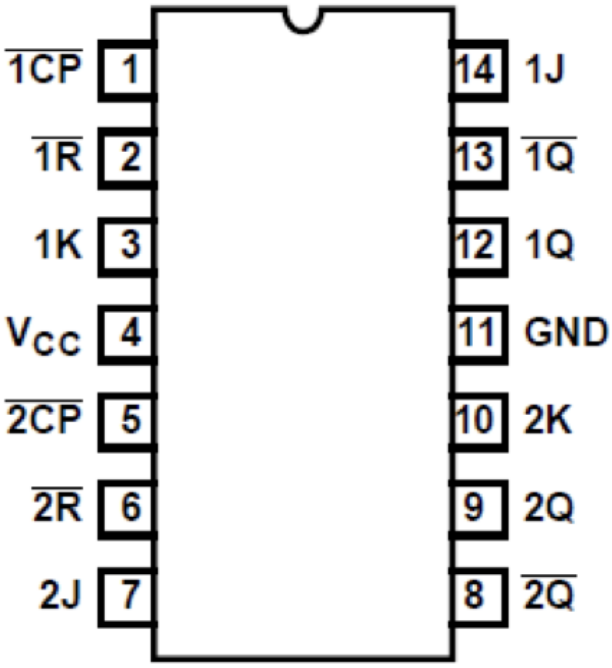

FIGURE 3.1: Pinout of the JK flip-flop.
Picture from Texas Instruments datasheet.

3.2.2 Counter

The CMOS Presettable BCD Up/Down Counter (CD4510B) is used as the counter. The pinout is shown in figure 3.2. Pin 16 is connected to power and pin 8 is connected to ground. The four Q inputs are on pin 2, 14, 11, and 6. The clock input is on pin 15, the up/down input is on pin 10, the preset enable input is on pin 1, and the reset input is on pin 9. Since the counter should always begin from zero, reset is engaged before the clock is started. Furthermore, the preset enable is connected to ground as it is not used.

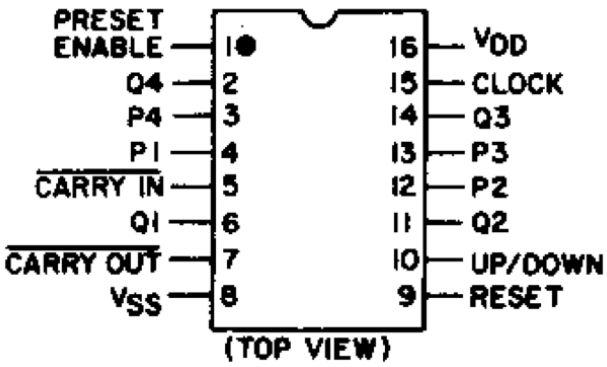

FIGURE 3.2: Pinout of the CD4510B.
Picture from Texas Instruments datasheet.

3.3 Diagram

When looking at table 3.1 it can be deduced that there are only two states, being 0 and 9, where there will be applied any logic to the counter input. When the state is 0, J is set to 1 while K is don't care. This means that the logic for J will be,

$$\bar{Q}_1\bar{Q}_2\bar{Q}_3\bar{Q}_4 \tag{3.1}$$

, which equates to:

$$\overline{Q_1 + Q_2 + Q_3 + Q_4} \tag{3.2}$$

Similarly K is set to 1 for state 9, meaning that the logic for K will be,

$$Q_1\bar{Q}_2\bar{Q}_3Q_4 \tag{3.3}$$

, which is the same as:

$$\overline{\bar{Q}_1 + Q_2 + Q_3 + \bar{Q}_4} \tag{3.4}$$

The reason the De Morgan rule is used above is to get a simpler circuit, but an even simpler circuit is obtainable, if a k-map was to be used. Though that has not been done, so these are the final equations.

Logic gates were then implemented into a logical circuit based on equations 3.1 and 3.2, which was then connected to the rest of the circuit making the counter. The final design can be seen in figure 3.3, where it ends with the anode seven segment display, which will show the output.

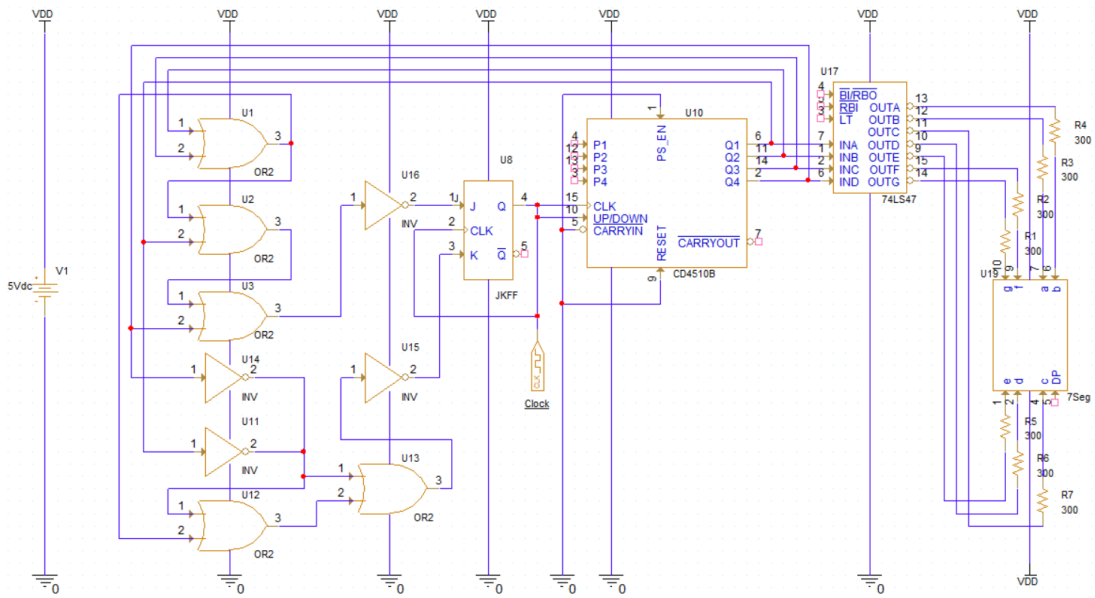

FIGURE 3.3: Diagram of the counter from ORCAD.

3.4 Simulation

To test the validity of the circuit design from 3.3, a simulation was made in the OrCAD program. The simulation was made by hooking the circuit up to a clock and then setting the preset value to 0000, making it count from 0 and up to start with (the same as enganging reset as described previously). Since the digital clock component is used, a fully digital graph was made, which is seen in the figure below. The figure shows that the values count up to 9 or 1001 (where LSB is Q1 and MSB is Q4) as expected, and then decrements back down to 0 or 0000. The simulation matches the truth table and expectations perfectly, meaning that the design should work in practice, as long as it is implemented correctly.

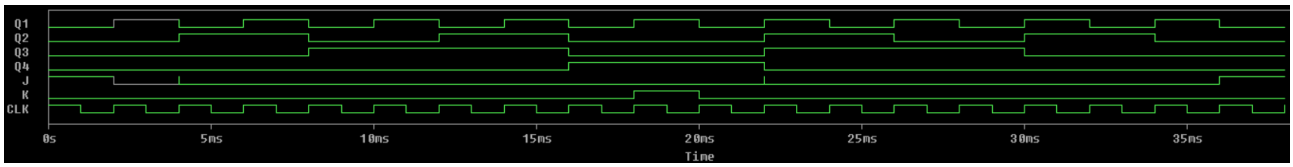

FIGURE 3.4: Simulation of the counter from OrCAD.

3.5 Implementation

Since everything fit together and worked out in theory, the design from figure 3.3 was made into a counter using the components described earlier. To test if the counter worked, you simply had to reset the counter, start the clock and observe the display. The counter can be seen working in the attached video file. However, one of the components (BCD display converter) did not quite fit in the breadboard, leading to one of us having to press on it with a finger in the video.

4 Conclusion

In conclusion, we have successfully designed and implemented a digital counter with a 7-segment display. The counter works by counting from 0 to 9, and then back to 0 in a loop, with one number counted per clock cycle. With that knowledge a truth table is set up, which leads to a boolean expression that can be implemented. The counter is implemented using a synchronous counter alongside OR gates, NOT gates, and a JK flip-flop. Everything worked as intended both in a digital simulation made in OrCad as well as in real life, when it was implemented. The attached video shows the practical implementation of the counter working as described.